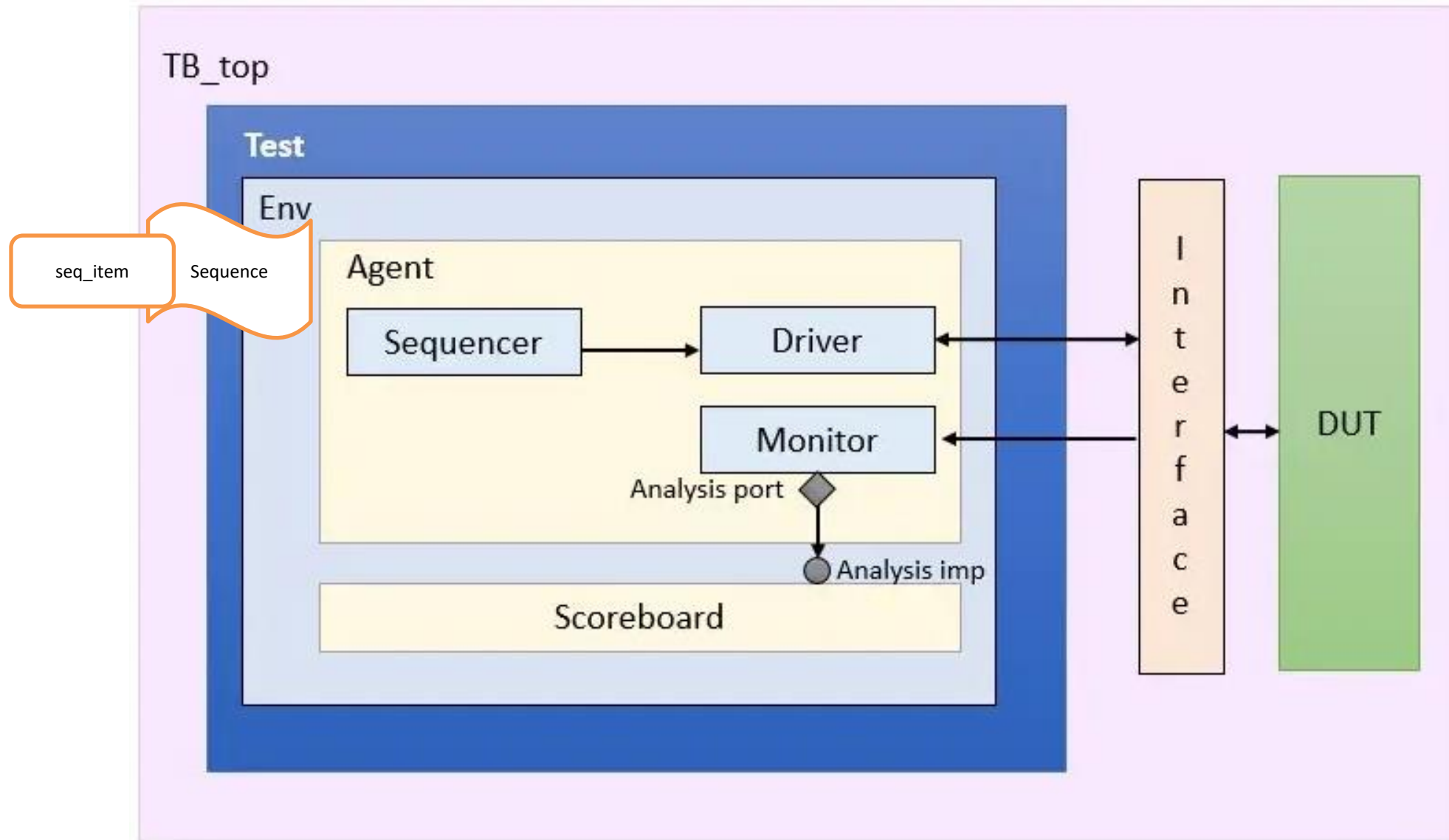


UVM Driver, Monitor and Agent

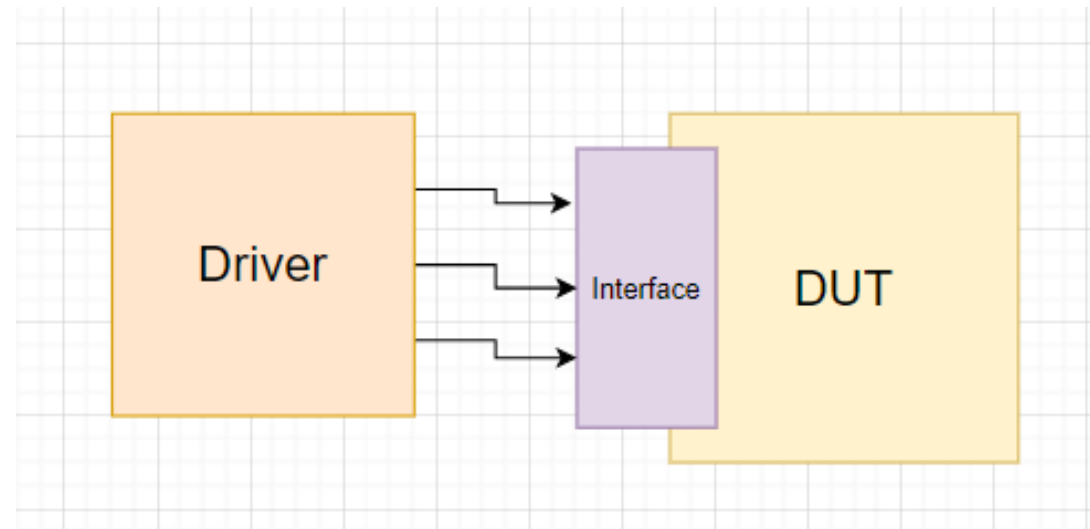
Yasir Farooq

Recall the UVM based Testbench



Driver

- The UVM driver is an active entity that has knowledge on how to drive signals to a particular interface of the design
- Always extended from *uvm_driver* base class
- Transaction-level objects are obtained from sequencer and the driver drives them it to the design via an interface handle



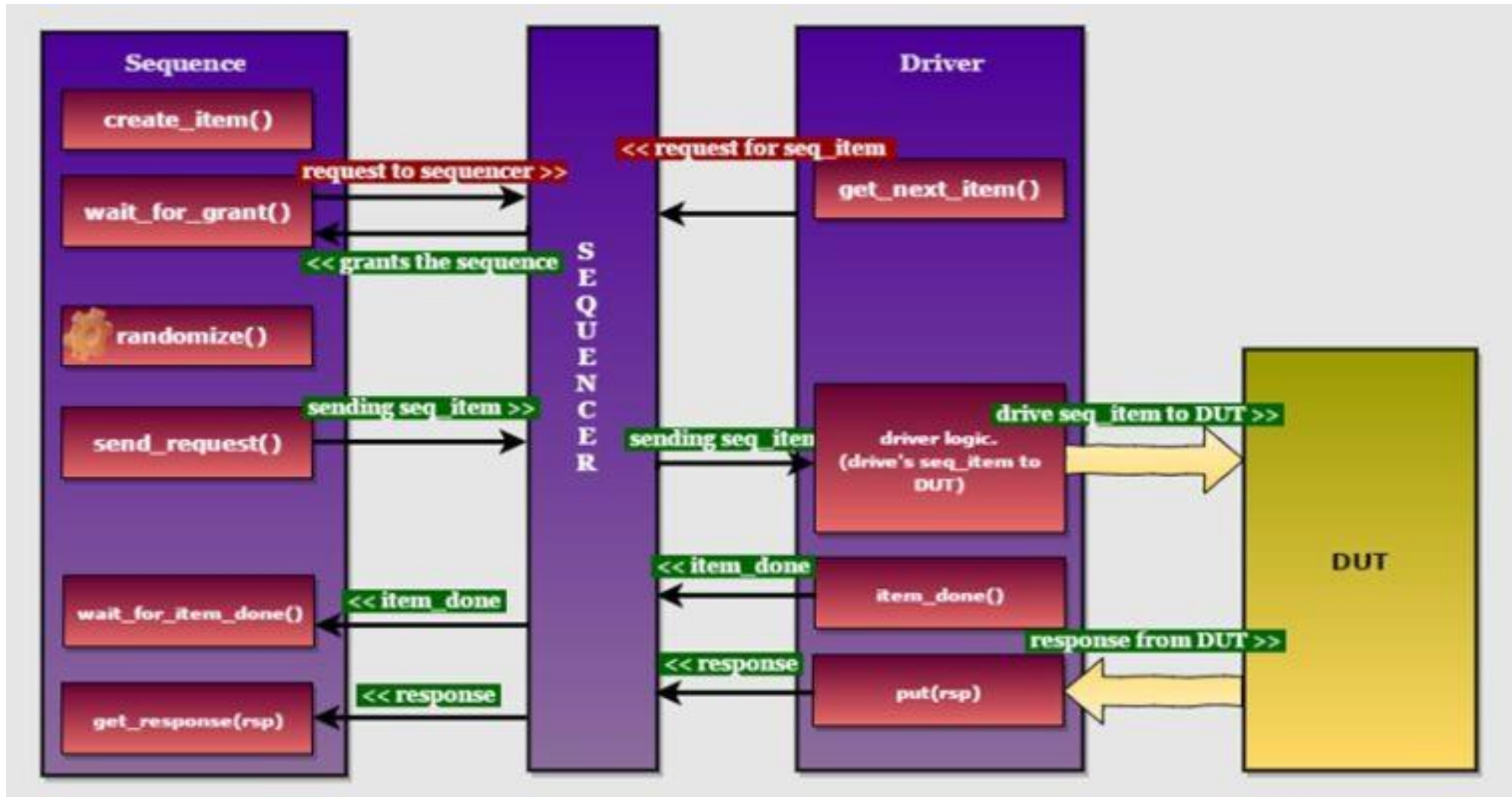
Driver – Methods

UVM driver communicates with sequencer via the following methods:

- **get_next_item():** This method blocks until a REQ sequence_item is available in the sequencer
- **try_next_item():** This is a non-blocking variant of the get_next_item() method. It will return a null pointer if there is no REQ sequence_item available in the sequencer.
- **item_done():** The non-blocking item_done() method completes the driver-sequencer handshake, and it should be called after a get_next_item().
- **put():** The put() method is non-blocking and is used to place an RSP sequence_item in the sequencer.

Sequence-Sequencer-Driver Handshake

- With DUT response



Driver – Example Code

```
class alu_driver extends uvm_driver#(alu_transaction);
  `uvm_component_utils(alu_driver)
  // Interface Declaration
  virtual tinyalu_intf alu_if;

  // Constructor Define Method
  function new (string name, uvm_component parent);
    super.new(name,parent);
  endfunction

  // Build Phase
  function void build_phase (uvm_phase phase);
    super.build_phase(phase);
    // Add UVM Macros
    if(!uvm_config_db#(virtual tinyalu_intf)::get(this, "", "alu_if", alu_if))
      `uvm_fatal("NO_VIF",{ "virtual interface must be set for: ",get_full_name(),".aluIF"})
    endfunction

  // Run Phase
  task run_phase (uvm_phase phase);
    alu_transaction alu_sq;
    reset_signals();
    @(posedge alu_if.clk);
    forever begin
      @(posedge alu_if.clk iff (alu_if.reset))
        seq_item_port.get_next_item(alu_sq);
        drive(alu_sq);
        seq_item_port.item_done();
      end
    endtask
```

Driver – Example Code (cont'd)

```
task reset_signals();
    wait(alu_if.reset)
        alu_if.A <= 'x;
        alu_if.B <= 'x;
        alu_if.start <= 1'b0;
        alu_if.op <= 2'bx;
endtask

virtual task drive (input alu_transaction alu_sq );
    @(posedge alu_if.clk && (alu_if.reset==1'b1));
    alu_if.start <= 1'b1;
    alu_if.A <= alu_sq.A;
    alu_if.B <= alu_sq.B;
    alu_if.op <= alu_sq.op;
    @(posedge alu_if.clk);
    alu_if.start <= 1'b0;
    @(posedge alu_if.done);
endtask
endclass
```

UVM Components- Definition

- A uvm_component class is the foundation of testbench structure
- Defining a UVM component
 - Extend the uvm_component class
 - Use the `uvm_component_utils() to register the class with factory
 - Use the uvm_component constructor
 - Override the uvm phase methods (if needed)

```
class scoreboard extends uvm_component;           // Extending a UVM component
    `uvm_component_utils(scoreboard);              // Registering the class with factory

    function new (string name, uvm_component parent); // Using the UVM constructor
        super.new(name, parent);
    endfunction : new
```


UVM Components- Driver

- Driver gets data items (sequences) from sequencer and drives the DUT
- Defining a Driver
 - Extend the **uvm_driver** class & parameterize it to work with sequence item
 - uvm_driver is derived from uvm_component, it inherits driver functions
 - It will be able to interact with the uvm_sequencer to get sequences
 - It will inherit seq_item_port, and all its functionalities
 - Use the ``uvm_component_utils()` to register the driver class with factory
 - It implement utilities for printing, copying and so on
 - Declare a virtual interface to connect the driver to the DUT
 - Use the uvm_component constructor
 - Modify the run_phase to use seq_item_port with its methods
 - get_next_item method blocks until the sequencer puts the data in port
 - Then, we can have user defined task to drive the DUT e.g. send_op
 - item_done method tells the sequencer that it can send another item

UVM Components- Driver (Example)

```
class driver extends uvm_driver #(sequence_item);           // Extending with uvm_driver
    `uvm_component_utils(driver)                             // Registering in factory
    virtual tinyalu_bfm bfm;                                 // Declaring a virtual interface
    function void build_phase(uvm_phase phase);
        // Getting interface from uvm_config_db
        if(!uvm_config_db #(virtual tinyalu_bfm)::get(null, "*", "bfm", bfm))
            `uvm_fatal("DRIVER", "Failed to get BFM")
    endfunction : build_phase
    task run_phase(uvm_phase phase);
        sequence_item cmd;                                  // Declaring a sequence item
        forever begin : cmd_loop
            shortint unsigned result;
            seq_item_port.get_next_item(cmd);                // blocking method, receives sequence items
            bfm.send_op(cmd.A, cmd.B, cmd.op, result);        // user defined task to drive the DUT
            cmd.result = result;
            seq_item_port.item_done();                        // method, telling that driver is ready for
        next item
        end : cmd_loop
    endtask : run_phase
    function new (string name, uvm_component parent);
        super.new(name, parent);                            // constructor
    endfunction : new
endclass : driver
```

UVM Components- Monitor

- **Monitor extract DUTs signal information and translate them into meaningful information (transaction) that other components can use**
 - Monitor may extract input signals of DUT and pass them to coverage collector for functional coverage analysis and, also to the scoreboard for predicting the result for comparison with actual result
 - Monitor may extract output signals of DUT for protocol adherence or to see results
- **Generally, Monitor consists of build and run phase. However, there are examples where no run phase is implemented and only user define methods are implemented along with build method**

UVM Components- Monitor

- **Defining a Monitor (Single Monitor with Run Phase)**
 - Extend the **uvm_monitor** class
 - uvm_monitor is derived from uvm_component, it inherits monitor functions
 - It will be able to interact with the DUT to get signal information through virtual interface
 - As monitor may need to broadcast the transaction to multiple components, for this uvm_analysis_port is used
 - Use the ``uvm_component_utils()` to register the monitor class with factor
 - It implement utilities for printing, copying and so on
 - Declare a virtual interface to connect the monitor to the DUT
 - Use the uvm_component constructor
 - Modify the run_phase to use uvm_analysis_port with its methods
 - Extract the I/O signals of DUT and translate them into valid transaction
 - After that use .write method to write the valid transaction on uvm_analysis_port so that the consumer part (scoreboard, coverage collector), can get this information through uvm export

UVM Components- Monitor (Example)

```
class alu_monitor extends uvm_monitor;                                // Extending with uvm_monitor
    `uvm_component_utils(alu_monitor)                                // Registering in factory

    virtual tinyalu_intf alu_if;                                    // Declaring a virtual interface
    uvm_analysis_port #(alu_transaction) mon_alu_before;           // Declaring UVM Analysis Port
    alu_transaction alu_sq;

    function new (string name,uvm_component parent);               // Constructor
        super.new(name,parent);
    endfunction

    function void build_phase (uvm_phase phase);                    // Build Phase
        super.build_phase(phase);
        mon_alu_before = new (.name("mon_alu_before"),.parent(this));
        if(!uvm_config_db#(virtual tinyalu_intf)::get(this, "", "alu_if", alu_if))
            `uvm_fatal("NO_VIF",{ "virtual interface must be set for: ",get_full_name(),".aluIF"})
    endfunction

    task run_phase (uvm_phase phase);                                // Run Phase
        alu_sq= alu_transaction::type_id::create(.name("alu_sq"));
        forever begin
            @(posedge alu_if.clk);
            wait (alu_if.done);
            alu_sq.result = alu_if.result;                            // Converting DUT Output Result to meaningful transaction
            mon_alu_before.write (alu_sq);                            // Calling ".write" method to write on mon_alu_before port
        end
    endtask
endclass
```

UVM Components- Agent

- Agent is collection of uvm_components focused around specific pin-level interface
- Typically instantiates driver, sequencer, monitor and connects them
- Defining an Agent
 - Extend the **uvm_agent** class
 - uvm_agent is derived from uvm_component, it inherits agent functions
 - It provides is_active data member which may have UVM_ACTIVE or UVM_PASSIVE value (active by default), and get_is_active method
 - For multiple agents, a config class may be used to pass the handle to the config, rather than virtual interface and value of is_active member, in the build phase
 - Example in next slide has one agent, so it doesn't need config class
 - Use the ``uvm_component_utils()` to register the agent class with factory
 - Use the uvm_component constructor
 - Use the connect phase to make connections

UVM Components- Agent (Example)

```
class alu_agent extends uvm_agent;           //Extends with uvm_agent
    alu_driver driver;
    alu_monitor monitor;
    alu_sequencer sequencer;
    `uvm_component_utils(alu_agent)           //Registering in factory
    function new (string name, uvm_component parent); //Constructor
        super.new (name,parent);
    endfunction

    function void build_phase (uvm_phase phase);           //Build phase
        super.build_phase(phase);
        monitor = alu_monitor::type_id::create ("monitor", this); //Instantiating monitor
        if (get_is_active() == UVM_ACTIVE //Conditional instantiation of driver, sequencer

        begin
            driver = alu_driver::type_id::create ("driver", this);
            sequencer = alu_sequencer::type_id::create ("sequencer", this);
        end endfunction

    function void connect_phase (uvm_phase phase);           //Connect phase
        super.connect_phase(phase);
        if (get_is_active() == UVM_ACTIVE) //Conditional connection of driver and sequencer
            begin driver.seq_item_port.connect(sequencer.seq_item_export); end
        endfunction
endclass
```

Lab Exercise

Lab 8 Using UVM Driver, Monitor and Agent

To understand driver and monitor classes in UVM